

Challenge 1.1: SQL Generation

Using only zero-shot prompting (no examples), write a prompt that generates a BigQuery SQL query to:

1. Find the top 5 departments by average salary
2. Include the department name, average salary, and employee count
3. Only include departments with more than 10 employees
4. Order by average salary descending

Prompt:

Create a BigQuery SQL query to find the top 5 departments by average salary.
Only include the department name, average salary, and employee count.
Only include departments with more than 10 employees.
Order by average salary descending.

Response:

```
SELECT
  department_name,
  AVG(salary) AS average_salary,
  COUNT(*) AS employee_count
FROM employees
GROUP BY department_name
HAVING COUNT(*) > 10
ORDER BY average_salary DESC
LIMIT 5;
```

What it does:

AVG(salary) → calculates average salary per department

COUNT(*) → counts employees per department

GROUP BY department_name → aggregates by department

HAVING COUNT(*) > 10 → only departments with more than 10 employees

ORDER BY average_salary DESC → highest paid first

LIMIT 5 → top 5 departments

If your table uses IDs instead (like department_id) and you need to join to a departments table, use this version:

```
SELECT
  d.department_name,
  AVG(e.salary) AS average_salary,
  COUNT(*) AS employee_count
FROM employees e
JOIN departments d
  ON e.department_id = d.department_id
GROUP BY d.department_name
HAVING COUNT(*) > 10
ORDER BY average_salary DESC
LIMIT 5;
```

Evaluation:

ChatGPT was able to generate an accurate response to what I wanted, even without the table structure it was able to fill in the blanks. If I were to use this query, all I would need to do is either tell ChatGPT to refine the SQL with proper column names or do it myself.

Score:

5

Challenge 1.2: Error Explanation

Write a zero-shot prompt that asks the LLM to explain this error message in plain English and suggest a fix:

```
google.api_core.exceptions.BadRequest: 400 Syntax error:
Expected end of input but got keyword SELECT at [3:1]
```

Prompt:

Explain this error to me, where it comes from, why it might happen, and possible solutions:

```
google.api_core.exceptions.BadRequest: 400 Syntax error:
Expected end of input but got keyword SELECT at [3:1]
```

Response:

This error is coming from BigQuery's SQL parser. It means your query has multiple SELECT statements or extra tokens, and BigQuery expected the query to be finished but found another SELECT.

google.api_core.exceptions.BadRequest: 400 Syntax error:
Expected end of input but got keyword SELECT at [3:1]

What the parts mean

400 BadRequest → BigQuery rejected the SQL syntax

Expected end of input → BigQuery thought the query was finished

but got keyword SELECT → It encountered another SELECT

[3:1] → Line 3, character 1 (start of line 3)

So your query probably looks like:

```
SELECT ...
```

```
FROM table
```

```
SELECT ...
```

```
FROM other_table
```

BigQuery does not allow two SELECTs in one query unless you connect them.

Why this happens

Common causes:

1. Two SELECT statements not combined

 Invalid

```
SELECT department_id
```

```
FROM department
```

```
SELECT salary
```

```
FROM employees
```

✓ Fix — use UNION ALL

```
SELECT department_id  
FROM department
```

```
UNION ALL
```

```
SELECT salary  
FROM employees
```

2. Missing comma before another SELECT (CTE mistake)

✗ Invalid

```
WITH a AS (  
  SELECT * FROM table1  
)  
SELECT * FROM table2
```

This is actually valid — but this isn't:

✗ Invalid

```
WITH a AS (  
  SELECT * FROM table1  
)  
SELECT * FROM a  
SELECT * FROM table2
```

You need to combine them.

3. Trailing SELECT after query accidentally

Very common when editing:

```
SELECT department_id  
FROM department
```

WHERE revenue > 0

SELECT *

4. Copy/paste mistake in Python string

Example:

```
query = """  
SELECT *  
FROM table1
```

```
SELECT *  
FROM table2  
"""
```

5. Missing parentheses in subquery

 Invalid

```
SELECT *  
FROM  
SELECT * FROM table
```

 Fix

```
SELECT *  
FROM (  
    SELECT * FROM table  
)
```

How to debug quickly

Look at:

[3:1]

That means line 3 starts with SELECT, and that's the problem.

Example:

```
1 SELECT *  
2 FROM table  
3 SELECT * <-- error here
```

Most likely in your case

Since your last question involved BigQuery + top 5 departments, you probably wrote something like:

```
SELECT department, AVG(salary)  
FROM employees  
GROUP BY department
```

```
SELECT *  
FROM something_else
```

or accidentally duplicated a query.

Fix patterns

Option 1 — Use CTE

```
WITH dept AS (  
  SELECT department, AVG(salary) avg_salary  
  FROM employees  
  GROUP BY department  
)  
SELECT *  
FROM dept
```

Option 2 — Use subquery

```
SELECT *  
FROM (  
  SELECT department, AVG(salary)  
  FROM employees  
  GROUP BY department  
)
```

Option 3 — Use UNION

SELECT ...

UNION ALL

SELECT ...

If you paste your query, I can point to exactly where the extra SELECT is.

Evaluation:

ChatGPT gave me a very in depth analysis on what most likely happened, with the issue being most likely a missed semi-colon. It was able to tell the error came from BigQuery, and suggested either using a semi colon or a UNION ALL statement. It also correctly identified that the response code 400 meant the query was rejected.

Score:

5

Challenge 1.3: Data Dictionary Entry

Write a zero-shot prompt to generate a data dictionary entry for a column called customer_lifetime_value in a dim_customer table. The entry should include: data type, description, business definition, calculation method, and example values.

Prompt:

Create a data dictionary entry for a column called customer_lifetime_value in the table dim_customer. The entry should include data type, description, business definition, calculation method, and example values.

Response:

Here's a clean data dictionary entry for customer_lifetime_value in dim_customer:



Table: dim_customer



Column: customer_lifetime_value

Data Type:

DECIMAL(12,2) (or NUMERIC(12,2) depending on warehouse)

Description:

Total estimated revenue a customer is expected to generate over the entire duration of their relationship with the business.

Business Definition:

Customer Lifetime Value (CLV) represents the monetary value of a customer based on historical purchases and/or predictive modeling. It is used to evaluate customer profitability, guide marketing spend, and support segmentation strategies.

Calculation Method:

Depending on business logic, typically calculated as:

Simple historical model:

$\text{customer_lifetime_value} = \text{SUM}(\text{order_amount})$

over all completed orders for the customer.

Predictive model (common approach):

$\text{customer_lifetime_value} =$
 $\text{average_order_value}$
 $\times \text{purchase_frequency}$
 $\times \text{average_customer_lifespan}$

Where:

$\text{average_order_value} = \text{total_revenue} / \text{total_orders}$

$\text{purchase_frequency} = \text{total_orders} / \text{active_time_period}$

$\text{average_customer_lifespan} = \text{avg months customer remains active}$

Example Values:

1250.50

0.00

342.99

9875.25

Notes (optional but common):

Includes only completed/paid transactions

Excludes refunds (unless net CLV is used)

Updated nightly during customer dimension refresh

Currency: USD 

If you'd like, I can also:

add source columns

define ETL logic

create SQL implementation

format for dbt docs / YAML

Evaluation:

ChatGPT could have had more helpful data inside of the response, but overall I think that it would help a new team member. I could have been more specific by providing some context.

Score:

4

Challenge 2.1: Column Name Standardization

You need the LLM to convert column names from various formats to snake_case following your team's convention.

Provide these examples in your prompt:

CustomerFirstName -> customer_first_name

order-total-amount -> order_total_amount

PRODUCT ID -> product_id

Then ask it to convert:

ShippingAddress Line1

total_Revenue_YTD

customerEmailAddress

ORDER__STATUS__CODE

Prompt:

I need you to convert some column names to snake_case. Here are some examples of what it should look like:

CustomerFirstName -> customer_first_name

order-total-amount -> order_total_amount

PRODUCT ID -> product_id

Please convert these column names in a concise way:

ShippingAddress Line1

total_Revenue_YTD

customerEmailAddress

ORDER__STATUS__CODE

Response:

Here are the snake_case conversions:

ShippingAddress Line1 → shipping_address_line1

total_Revenue_YTD → total_revenue_ytd

customerEmailAddress → customer_email_address

ORDER__STATUS__CODE → order_status_code

Evaluation:

Great response, not much could be done to improve it.

Score:

5

Challenge 2.2: SQL to Documentation

Teach the model to generate documentation from SQL using examples.

Prompt:

I need you to create documentation for SQL statements. Here is an example SQL statement with its example documentation.

```
SELECT customer_id, COUNT(*) as order_count  
FROM orders GROUP BY customer_id
```

Query: Customer Order Count

Purpose: Counts the total number of orders per customer

Tables Used: orders

Output Columns: customer_id, order_count

Aggregation: COUNT grouped by customer_id

Using the same format, can you document this query?

```
SELECT p.category,  
       DATE_TRUNC(o.order_date, MONTH) as month,  
       SUM(o.quantity * p.unit_price) as revenue,  
       COUNT(DISTINCT o.customer_id) as unique_customers  
FROM orders o  
JOIN products p ON o.product_id = p.product_id  
WHERE o.order_date >= '2024-01-01'  
GROUP BY 1, 2  
ORDER BY 1, 2
```

Response:

Query: Category Monthly Revenue and Customers

Purpose: Calculates monthly revenue by product category and counts distinct customers purchasing in each category

Tables Used: orders, products

Output Columns: category, month, revenue, unique_customers

Aggregation:

SUM of (quantity × unit_price) grouped by category and month
COUNT DISTINCT customer_id grouped by category and month
Joins: orders joined to products on product_id
Filters: order_date >= '2024-01-01'
Grouping: category, month (DATE_TRUNC to month)
Ordering: category, month

Evaluation:

ChatGPT was able to come up with a good response, although there may have been a bit too much information in the aggregation section. I could have given it constraints for the aggregation section for a more concise response.

Score:

4.5

Challenge 3.1: Cost Estimation

Use chain-of-thought prompting to have the LLM estimate BigQuery costs:

Scenario:

You have a table with 500 million rows, each row is approximately 500 bytes
You run a query that scans 3 columns (out of 20 total columns)
The query runs 4 times per day
BigQuery charges \$6.25 per TB scanned
Assume columnar storage distributes data evenly across columns